INFOSYS · FINAL PROJECT

CoachAI

Q&A Bank

25 Questions with Model Answers, Examples & Pro Tips
For the Mock (Milestone 4) and Final Presentation

Project	Repo	Date
AI-Powered Customer Support Assistant with Live Response Guidance	github.com/MasterJi27/coachai-infosys-final	August 2026

How to use this bank: Read once fully. Then rehearse **out loud** — 30 to 45 seconds per answer. Q&A; handlers: Handler 1 takes Q2–Q13 (technical), Handler 2 takes Q14–Q26 (product, deployment, team). If you don't know, say: *"Great question — let me verify in the repo and get back to you."* Never invent numbers.

Strictly 10 slides in the deck · Bullet points only · Professional fonts · Screenshots where relevant

Contents

Quick reference: Full KT guide → **Doc/CoachAI_Knowledge_Transfer.md** · Browser docs → **Doc/CoachAI_Project_Documentation.html** · Slides → **Doc/CoachAI_Presentation.pptx** · Deep dive → **Doc/CoachAI_Deep_Dive.md**

01 What is CoachAI and what problem does it solve? — Project Overview

ANSWER — what to say (30–45 sec)

CoachAI is an **AI coaching cockpit** for support agents. Traditional QA happens **after** the call — feedback arrives days later, escalations are caught too late, knowledge is scattered, and quality depends on the individual agent. CoachAI moves coaching **into the moment**: every customer message is analyzed in <1 second for sentiment, intent, the right KB article, a suggested reply, and risk signals (CSAT, churn, fraud, viral) — before the agent hits send. After the session it auto-generates a performance report.

EXAMPLE

Customer: *"My order ORD-8142K not received, I want refund NOW!"* → ***Coach shows:*** *Intent = refund_request, Sentiment = angry, Frustration 82%, Escalation 78%, KB = "Refund — Undelivered Orders", Suggested reply = empathetic refund steps, Risk = high — manager whisper triggers.*

Pro tip: *Start with the 30-sec elevator pitch, then give the ORD-8142K example. End with "We coach in the moment, not after the call."*

02 What is a Multi-Agent System and why not a single big prompt? — Multi-Agent System

ANSWER — what to say (30–45 sec)

One agent = one responsibility. We have **25+ agents** — classifier, coach, monitor, auditor, etc. — coordinated by an **Orchestrator**. This makes prompts small and testable, lets agents run **in parallel** (Intent + Deep Analysis together), allows **adaptive behavior** (Calibrator learns per agent when to show tips), and gives **fault isolation** — if fraud detection fails, coaching still works. A single mega-prompt would be slow, brittle, and impossible to debug or evaluate.

EXAMPLE

Per turn: ***IntentSentimentAgent*** *+* ***DeepAnalysisAgent*** *run in parallel →* ***KnowledgeRecommendationAgent*** *→* ***CoachingSuggestionAgent*** *→* ***EscalationMonitorAgent****. All outputs merge into one* ***TurnAnalysis****.*

Pro tip: *If they ask "Isn't it over-engineering?" — say: at FAQ scale it is intentionally modular; for production you can collapse 2-3 agents, but evaluation stays per-agent.*

03 How many agents are there? Name the key ones. — Multi-Agent System

ANSWER — what to say (30–45 sec)

25+ agents in **src/agents/**. Key groups: **Core pipeline** — customer_simulator, intent_sentiment, knowledge_recommendation, coaching_suggestion, escalation_monitor, deep_analysis, post_interaction_summary. **Quality** — compliance_monitor, tone_rewriter, qa_audit_agent, coach_calibrator. **Risk** — predictive_csat, viral_threat_predictor, fraud_detector, competitor_defection_agent, customer_mind_reader, multiverse_simulator. **Tools** — auto_pilot_agent, jira_bug_generator, scenario_generator, bot_agent. Plus cognitive_load and patience_clock.

EXAMPLE

Demo flow: show ***Feature Lab*** *in the React Dashboard — Multiverse, Tone Rewriter, Compliance, Jira Ticket — each is one agent.*

> ***Pro tip:*** *Don't try to name all 25 in order. Name 5-6 core + 2-3 risk agents; say "full table is in README and Doc/Deep_Dive.md".*

04 Explain the per-turn pipeline — what happens after a customer message arrives? — Architecture

ANSWER — what to say (30–45 sec)

9 steps: 1) Customer message arrives (typed/simulated/replayed). 2) Orchestrator → ConversationManager. 3) **IntentSentimentAgent** (LLM) → intent, sentiment, frustration, trend. 4) **KnowledgeRecommendationAgent** → hybrid KB search + LLM tip (this is the **RAG** step). 5) **CoachingSuggestionAgent** → reply + tone/clarity. 6) **EscalationMonitorAgent** → rule-based risk score. 7) **DeepAnalysisAgent** (parallel) → CSAT, churn, viral/PR, fraud, mind-read. 8) Merge into **TurnAnalysis** → render in coaching console. 9) On session end, **PostInteractionSummaryAgent** builds the report.

EXAMPLE

Turn 2: Customer "Where is my refund?" → Intent=billing_dispute, Frustration 65% → KB="Refund Timeline 5-7 days" → Coaching suggests empathetic timeline + tracking steps → Escalation 55% → Risk rail updates instantly.

> ***Pro tip:*** *Draw the 4-tab rail on the board:* ***Signals*** *→* ***KB*** *→* ***Reply*** *→* ***Risk*** *— that is the pipeline rendered.*

05 What is RAG and how did you implement retrieval and generation? — RAG

ANSWER — what to say (30–45 sec)

Retrieval-Augmented Generation = retrieve docs, then generate. Retrieval: hybrid search — **semantic** (OpenRouter embeddings **nvidia/nemotron-3-embed-1b:free**, 2048-dim, cosine) as primary, **keyword/BM25** overlap as offline fallback; embeddings cached as JSONB. **Generation:** LLM synthesizes **one distilled coaching tip** from the top hit. If top relevance < 0.45 we flag a **KB gap** and trigger Auto-KB to draft a new FAQ.

EXAMPLE

Query: *"refund not received" →* ***Retrieved:*** *"Refund Policy — Undelivered Orders (content...)" score 0.82 →* ***Generated tip:*** *"As per policy, refunds for undelivered orders process in 5-7 business days; ask for UTR and escalate if >7 days."*

> ***Pro tip:*** *Why no vector DB? At 19 FAQs, in-memory + JSONB is instant and dependency-light. pgvector is the planned scale-up beyond FAQ scale.*

06 How does hybrid retrieval work and why hybrid? — RAG

ANSWER — what to say (30–45 sec)

Semantic catches meaning ("money back" matches "refund"), but needs an API key and cold-start embedding. **Keyword/BM25** always works offline and is instant. We try **semantic first** (lazy-embeds docs on first query), fall back to keyword if the key is missing or the call fails. Node backend stores embeddings in Postgres JSONB so warm instances are fast; Python backend keeps an in-memory list rebuilt from **data/knowledge_base/** on boot.

EXAMPLE

Without key: *query "UPI payment failed" → keyword overlap finds "UPI Troubleshooting" via token match.* ***With key:*** *same query → embedding cosine finds it even if wording differs ("transaction declined").*

> ***Pro tip:*** *If they ask about Chroma/FAISS/Pinecone — say we intentionally avoided it at FAQ scale; the hybrid gives the same result with zero infra.*

07 What happens when the knowledge base has no answer (KB gap)? — RAG

ANSWER — what to say (30–45 sec)

If top relevance < **0.45**, we emit a **"Knowledge Base Gap Detected"** item (source = kb-gap-alert) in the KB rail and log it in **knowledge_gaps** on the report. **Auto-KB Agent** then drafts a new FAQ JSON to **data/knowledge_base/pending/faq_auto_gen_*.json** with title, category, content, keywords, status = "Pending Review". A human reviews before publishing. This closes the loop: gaps found in live chats become new KB articles.

EXAMPLE

Customer asks about "GST invoice correction" — no KB hit above 0.45 → UI shows gap alert → Auto-KB drafts "FAQ: GST Invoice Correction — Verify against actual policy before publishing" → appears in KB Admin tab.

Pro tip: *This is a strong differentiator — mention it when they ask "How do you keep the KB fresh?"*

08 Which LLMs do you use and what is the fallback strategy? LLM Strategy

ANSWER — what to say (30–45 sec)

Priority order in src/core/llm.py and backend/src/llm.js: 1) **Azure OpenAI** (Responses API, e.g. gpt-5.4-mini) if AZURE_OPENAI_ENDPOINT/KEY are set — fastest, no queue. 2) **Race** OpenRouter free vs Groq free in a 2-thread pool — first non-empty wins. OpenRouter tries its fallback chain; Groq chain is **llama-3.3-70b-versatile** → **llama-3.1-8b-instant** → **gemma2-9b-it** → **llama3-8b-8192**. 3) Empty string → caller uses **hard-coded heuristic fallback**. Also: **embed_text()** (OpenRouter embeddings) and **text_to_speech()** (fish-audio neural TTS with gTTS fallback). Keys resolve from settings → env → st.secrets.

EXAMPLE

Azure down? Groq answers in ~1s. No keys at all? IntentSentiment falls back to keyword rules, coaching falls back to templated empathetic reply — demo still runs.

Pro tip: *Phrase it as "graceful degradation" — Infosys loves resilience. Say "The app never crashes without keys, it degrades."*

09 What are System Prompts and where are they stored? System Prompts

ANSWER — what to say (30–45 sec)

A **system prompt** is the instruction that tells an agent how to behave. Every LLM agent imports its prompt from **src/core/prompts.py** (15+ constants + 4 builder functions) — the **single source of truth**. The Node mirror is **backend/src/agents.js**. Builder functions interpolate per-turn context (persona, valid intents enum, retrieved KB article, macros library) into the prompt at call time. Agents must return **structured JSON** (e.g. {"reply":"...","tone":"empathetic","clarity":1-5}) parsed with tryJson/regex; failures use hard-coded fallbacks.

EXAMPLE

CoachingSuggestion prompt *is built as: "You are a support coach. Given customer message + KB article + sentiment, suggest the best reply as JSON {reply, tone, clarity, key_point}" — then we inject the live values.*

Pro tip: *Offer to open **src/core/prompts.py** live — it's the most impressive file to show prompt engineering.*

10 Can you show an example system prompt? System Prompts

ANSWER — what to say (30–45 sec)

Customer Simulator (Hinglish): instructs the LLM to generate a realistic customer message ≤140 chars with a sentiment state machine, and when Hinglish mode is on, to "aggressively mix Hindi and English in Latin script — e.g. 'mera account chal nahi raha hai', 'kya kar rahe ho yaar', 'sir please check karo na' — Tier-2/Tier-3 Indian customer." **Tone Rewriter:** "Rewrite the agent's draft to be warmer, clearer and more empathetic. Keep it professional and concise (max 4 sentences). Respond ONLY with the rewritten text." Short drafts (<10 chars) get a canned apology (rate-limit guard).

EXAMPLE

Input draft: *"unfortunately we can't refund"* → ***Tone Rewriter output:*** *"I completely understand your frustration. Let me help — your refund is eligible and will process within 5-7 business days."*

Pro tip: *If they ask to see a prompt, show the file and scroll — the Hinglish instruction always gets a reaction.*

11 How does Escalation Monitoring work? Is it LLM-based?

Risk & Quality

ANSWER — what to say (30–45 sec)

No — it is rule-based (no LLM) for speed and determinism. Weighted formula: **frustration×0.4 + sentiment +0.2 + trend +0.15 + keywords×0.1 + length +0.1 + repetition +0.05**. Keywords like "manager", "legal", "consumer court" boost the score. The rail shows a risk meter + strategy. At high risk the **Manager Supervisor agent** (LLM) whispers advice or takes over the chat.

EXAMPLE

Customer: "I want to talk to your manager, this is the third time!" → *keywords (manager, third time) + frustration 75%* → *escalation 82%* → *whisper: "[Manager] Escalation risk 82% — authorize the resolution now. Do not ask more questions."*

Pro tip: *Emphasize: escalation detection is **instant and offline** — no LLM latency when it matters most.*

12 How do you predict CSAT, churn, fraud and viral/PR risk?

Risk & Quality

ANSWER — what to say (30–45 sec)

DeepAnalysisAgent does it in **one LLM call** that returns a single JSON dict with: predicted CSAT (1-5) + churn %, viral/PR risk + pre-approved PR statement, fraud risk + category + protocol, defection risk + competitor + counter-offer, internal monologue, escalation trigger. In the Node backend these are split into focused agents (viralThreat, fraudAnalysis, defectionAnalysis, mindReader, multiverse, patienceAnalysis, cognitiveLoad) that run the same logic with keyword pre-scans ("twitter/x/consumer court" for viral, "hacked/stolen/unauthorized" for fraud, "switch to Swiggy/Zepto" for defection) seeding defaults before the LLM refines them.

EXAMPLE

"I'll post on Twitter and switch to Swiggy!" → *Viral High (twitter), Defection High (Swiggy), PR statement drafted, retention offer "STAY15" suggested, fraud Low.*

Pro tip: *Show the Risk tab in the Dashboard — it displays all 6-7 risk chips + PR statement + retention offer live.*

13 What is the Mind-Reader and Multiverse Simulator?

Risk & Quality

ANSWER — what to say (30–45 sec)

Mind-Reader infers the customer's **unstated true intent**: {true_intent, internal_monologue, emotional_state, unspoken_need}. Falls back to sentiment analysis if the LLM fails. **Multiverse Simulator** is a "what-if" tool: given the customer message it generates **3 alternative agent replies** with predicted outcome + escalation risk (e.g. "Refund first" → risk drops 50%, "Ask questions first" → risk rises). Both are LLM-based (temp 0.3 and 0.5) with heuristic fallbacks.

EXAMPLE

Mind-Reader: *Customer "Okay fine do whatever"* → *true_intent: "Wants the money back, not promises", monologue: "This is taking too long...", unspoken_need: "To be taken seriously."* ***Multiverse:*** *same turn* → *branch 1 "I'll refund now" (risk 18%), branch 2 "Can you share order ID again?" (risk 72%).*

Pro tip: *Run Multiverse live in Feature Lab — it's the most visual "wow" feature after the coaching rail.*

14 How does the frontend communicate with the backend?

Architecture

ANSWER — what to say (30–45 sec)

React frontend (Vite, port 5173) talks to **Node Express backend (port 8000)** via **REST + JSON** over fetch. A single wrapper in **frontend/src/lib/api.js** adds the bearer token from localStorage and handles errors. API base is **VITE_API_URL** or prod **https://coachai-backend-swart.vercel.app**. Vite proxies **/api** and **/health** to localhost:8000 in dev. The dashboard polls session state per turn; no websockets — simple request/response keeps it serverless-friendly.

EXAMPLE

Login: *POST /api/auth/login {email,password} → {user, token} → stored in localStorage → all later calls send Authorization: Bearer <token>.* ***Chat:*** *POST /api/chat/message {session_id, message} → {messages, last_turn}.*

Pro tip: *If they ask about real-time, say: polling per turn is sufficient at <0.5s; websockets are a future improvement for sub-100ms.*

15 How does authentication work and why only in the Node backend? Architecture

ANSWER — what to say (30–45 sec)

Node backend only implements **/api/auth/***: register, login, guest, logout, me. Passwords are hashed (hashPassword/verifyPassword), tokens are generated via **generateToken()** and stored in **coach_auth_sessions** with is_active + expiry; the frontend sends **Bearer token** and **authUser(req)** resolves it per request. The **Python FastAPI backend has no auth routes** — it's the legacy research stack. The frontend login only works against the Node backend, which is why the correct Vercel deployment directory matters (see gotchas). Guest login auto-creates a demo user.

EXAMPLE

*Click **Guest Demo** on the Landing page → POST /api/auth/guest → instant token → Dashboard opens without registration — perfect for the live demo.*

Pro tip: *This is the #1 gotcha to mention if they ask "Why two backends?" — auth is the reason the frontend needs Node.*

16 How is session state persisted across serverless cold starts? Architecture

ANSWER — what to say (30–45 sec)

Vercel functions are **ephemeral** — in-memory state is lost on cold start. We persist the **full SessionState as JSON** in Postgres (column **full_state** / **coach_sessions**) on every turn via **persistSession()**. On the next request, **getSession(session_id)** checks the in-process cache first, then **rehydrates** the full state from the DB. SQLite (**data/coach.db**) is the local fallback when DATABASE_URL is absent. Reports and hall-of-fame are also authoritative in the DB, never on disk.

EXAMPLE

Agent handles turn 5, instance dies, customer sends turn 6 to a new cold instance → new instance loads session 5-turn history from Postgres and continues without loss.

Pro tip: *Say "We learned this the hard way — never read reports_dir on Vercel, always the DB" — it shows you understand serverless.*

17 Why Postgres with SQLite fallback? Explain the database design. Architecture

ANSWER — what to say (30–45 sec)

Postgres (via **pg** in Node, **SQLAlchemy** in Python, pool_pre_ping, sslmode=require) is used when **DATABASE_URL** is set (Neon/Supabase/Azure). Tables: **sessions** (with full_state JSON for rehydration), **reports**, **knowledge_chunks**, **agent_calibration**, **coach_activity_log**. When DATABASE_URL is absent (local dev), we fall back to **SQLite at data/coach.db** or an ephemeral temp dir on Vercel (**coachai-runtime**). This keeps local setup zero-config while production is durable and multi-instance safe.

EXAMPLE

*Run **python run.py** with no DATABASE_URL → SQLite file appears at data/coach.db. Deploy to Vercel with DATABASE_URL → same code uses Postgres, no code change.*

Pro tip: *If they ask about pgvector — say embeddings are currently JSONB + cosine in JS; pgvector is the planned upgrade.*

18 What are mock tools vs real integrations (Composio)? Integrations

ANSWER — what to say (30–45 sec)

Mock backend (src/tools/mock_backend.py) simulates OMS/payment/loyalty: lookup_order (randomized demo data), process_refund (rejects >■500 without supervisor), grant_loyalty_voucher — returns ToolCallResult for demo. **Composio (composio_backend.py, v3 SDK)** promotes these to **real actions**: Jira JIRA_CREATE_ISSUE, Gmail GMAIL_CREATE_EMAIL_DRAFT/send, Slack post. Graceful degradation: missing key or unconnected account → failed ToolCallResult with a helpful message, no crash, no UI change.

EXAMPLE

Without Composio: *Auto-Pilot "Refund processed" is text only.* ***With Composio + Gmail connected:*** *same action drafts a real refund email via Gmail and posts to #support-ops on Slack.*

Pro tip: *Show the* ***Integrations status*** *endpoint or the Jira Ticket Generator in Feature Lab — it prints a COACH-XXXX ticket ID.*

19 How is the performance report generated and scored? Reports & Analytics

ANSWER — what to say (30–45 sec)

PostInteractionSummaryAgent (rule-based, no LLM) builds a **PerformanceReport** at session end. **Overall = resolution×0.5 + coaching×0.3 + clarity/CSAT×0.2**. Inputs: sentiment journey, resolution_quality, knowledge_gaps, escalation_triggers, coaching tips. The report is saved to the DB and optionally **emailed via Composio** (recipient_email on /api/chat/end). Cross-session **PerformanceAnalytics** aggregates trends; **Hall of Fame** archives score ≥0.85 (■), **Hall of Shame** ≤0.45 (■). Score history and top gaps feed the Analytics page.

EXAMPLE

Session with 8 turns, resolved, 3 coaching tips followed → resolution 0.9, coaching 0.8, clarity 0.85 → overall 0.87 → Hall of Fame. Response includes sentiment journey chart + knowledge gaps list.

Pro tip: *Open the* ***Reports*** *page live — show a report's sentiment journey and the "Email Report" flow.*

20 How do you evaluate quality? What is the QA Audit Agent? Risk & Quality

ANSWER — what to say (30–45 sec)

QA Audit Agent is a **rule-based, ISO-9001-style** pass/fail auditor. It checks **21 checks** (e.g. overall_score ≥ 0.75, issue_resolved, no escalation triggers, no compliance violations, tone/clarity thresholds). Returns {status: PASS/FAIL, checks_passed, total_checks: 21, issues[]}. It runs on demand via **POST /api/analysis/qa-audit** and is shown in Feature Lab. Complementary: **Compliance Monitor** flags replies that contradict KB policy (e.g. promising "refund immediately" without a KB match, using "unfortunately").

EXAMPLE

Audit a low-score report (0.42) → FAIL, 14/21 passed, issues: ["Resolution score critically low", "Issue was not resolved", "Escalation triggers: manager_request"].

Pro tip: *Run QA Audit live on a Hall of Shame session — the FAIL is dramatic and memorable.*

21 What are the three interaction modes and what is Survival Mode? Features

ANSWER — what to say (30–45 sec)

Interaction modes (InteractionMode enum) in session_config.py: 1) **Simulator** — AI customer (customer_simulator agent, Hinglish) generates the next message; best for training. 2) **Manual** — human types both sides. 3) **Replay** — replays a transcript from

data/transcripts/. **Survival Mode** is a gamified arcade: 4 random tickets from SCENARIO_POOL, keyword-graded replies (quality 0.85/0.6/0.35), HP/score/streak/power-ups (Manager Shield at 3-streak, Instant Refund Pass at 5), time penalties, all-resolved bonus +500. In the Node backend it is a module-level singleton (not session-scoped).

EXAMPLE

Survival ticket: "My biryani is cold, rider is late!" → reply must hit keywords [apologize, reorder/refund, ETA] → coverage 2/3 + time bonus → score 78/100 → next ticket appears.

Pro tip: *Play one Survival turn live — it's fun and shows the product has a training game, not just a console.*

22 How did you handle Hinglish and short realistic customer messages? System Prompts

ANSWER — what to say (30–45 sec)

The **Customer Simulator** prompt enforces: ≤**140 chars**, a **sentiment state machine** (angry/frustrated/neutral/satisfied), and when Hinglish mode is on, an **aggressive Hinglish instruction**: "You MUST mix Hindi and English in Latin script — 'mera account chal nahi raha hai', 'kya kar rahe ho yaar', 'sir please check karo na' — Tier-2/Tier-3 Indian customer." The simulator runs at **temp 0.8** for variety, with 6 fallback scenarios if the LLM fails. This makes training realistic for the Indian market.

EXAMPLE

Without Hinglish: *"My order is delayed, please check."* ***With Hinglish:*** *"Bhaiya mera order abhi tak nahi aaya, kya kar rahe ho yaar, jaldi check karo na!"*

Pro tip: *Mention this when they ask about localization — it's a concrete, memorable detail.*

23 Why do you have two backends sharing one Vercel project? Architecture

ANSWER — what to say (30–45 sec)

Historical evolution: **Streamlit-only** → **Python FastAPI** → **Python on Vercel + Postgres** → **React frontend** → **Node Express backend** (commit 9431569). The **Node backend** was added because the frontend needs **auth** (/api/auth/*) which the Python API doesn't have. Both **backend/** (Node) and **vercel-backend/** (Python, a byte-for-byte copy of src/) are linked to the same Vercel project **coachai-backend**. The Node one is what the frontend actually uses. This is the **#1 gotcha** — verify which directory Vercel deploys from before shipping, and keep the two copies in sync (53 files).

EXAMPLE

Deploy check: ***vercel --prod*** *from backend/ vs vercel-backend/ — wrong directory means login breaks because /api/auth/* is missing.*

Pro tip: *Be honest about this tech debt — say "We would merge into one canonical backend next" — maturity impresses evaluators.*

24 How did you test the system? Testing

ANSWER — what to say (30–45 sec)

Three suites: 1) **Python pytest** — 14 tests + 1 load script in **tests/** covering agents, RAG, and load. 2) **React vitest** — 7 tests in **frontend/src/test/** (ToastContext, ErrorBoundary, JiraBoard, CSV/PDF export). 3) **Node** — **node --test tests/** in backend. Run: **python -m pytest tests/**, **cd frontend && npm test**, **cd backend && npm test**. Beyond unit tests, we do **manual QA** via Simulator and Replay modes and check graceful degradation by running with no API keys.

EXAMPLE

Test with no GROQ_API_KEY: start a Simulator session → intent falls back to keyword rules, coaching to templated reply — suite still passes, demo still runs.

Pro tip: *If they ask about coverage, say "We test agents and RAG contract boundaries; LLM outputs are non-deterministic so we assert structure, not exact text."*

25 What are the limitations and what would you improve next? Future Work

ANSWER — what to say (30–45 sec)

Limitations: No vector DB at scale (in-memory + JSONB only), two backends to maintain, auth only in Node, survival game is a global singleton (not per-user), hard-coded demo data (■250, Biryani Blues), limited multilingual beyond Hinglish. **Next steps:** 1) Persist embeddings with **pgvector**. 2) Merge Node + Python into **one canonical backend**. 3) Add **voice/telephone coaching** and full multilingual support. 4) Team **leaderboards, gamified upskilling paths, HR analytics**. 5) Move deep_analysis JSON contract into a **Pydantic model** for type safety. 6) Reconcile README with code reality.

EXAMPLE

Scale story: "At 19 FAQs we are instant; at 10,000 docs we would switch to pgvector with HNSW indexing and pre-embedded chunks."

Pro tip: *Always end future-work answers with "The current design is intentionally dependency-light for FAQ scale — scaling is a clear next iteration."*

26 What was your individual contribution and what did you learn? Team & Learnings

ANSWER — what to say (30–45 sec)

Tailor this per person, but structure it as: **1) My module:** e.g. "I built the RAG pipeline / the React Dashboard / the Node auth + session store / the prompt repository." **2) Key learning:** "I learned **system prompt engineering** — how a single instruction file drives 25 agents; **multi-agent orchestration** — parallel vs sequential, calibrators, supervisors; **RAG** — hybrid retrieval with graceful degradation; **LLM fallback chains** and session rehydration on serverless." **3) Challenge:** "Hardest was keeping two backends in sync and making the app never crash without keys." See **Doc/CoachAI_Knowledge_Transfer.md Section 11** for the role playbook (2 PPT, 2 presenters, 2 Q&A;).

EXAMPLE

"I owned the Coaching Console rail — Signals/KB/Reply/Risk tabs. I learned how to map TurnAnalysis to UI chips and handle the race between semantic and keyword retrieval."

Pro tip: *Every team member must have a **different** contribution story — decide by Monday as the email says, and rehearse it.*

Closing & How to Handle Tough Questions

If you don't know the answer

Never guess numbers, model names, or scores. Use this exact line:

> *"That's a great question — I want to give you the accurate answer. Let me verify it in the repo (**github.com/MasterJi27/coachai-infosys-final**) and get back to you right after the session."*

Q&A handling playbook (2 handlers)

- **Handler 1 — Technical:** Q1–Q13 (multi-agent, RAG, LLM, prompts, risk). Keep **src/core/prompts.py** open on your laptop.
- **Handler 2 — Product & Deployment:** Q14–Q26 (architecture, auth, DB, integrations, reports, modes, testing, future, team). Keep the **live demo** and **Reports page** open.
- One person answers, the other adds a one-line example if needed. Don't both talk at once.
- Always end an answer by **pointing to the repo or the live UI** — it shows confidence and evidence.
- Timebox: **30–45 seconds** per answer. If they want more, they'll ask a follow-up.

Final line to close Q&A

> *"Thank you for your questions and feedback. The full code, 10-slide deck, and documentation are available at **github.com/MasterJi27/coachai-infosys-final**. We welcome your suggestions."*

Good luck — you've got this. ■